

LowMachReact-Hex

Technical Solver Report

Version 0.5.0

LowMachReact-Hex Development Notes

May 2026

Abstract

LowMachReact-Hex is a Fortran 2008 MPI finite-volume solver for hexahedral meshes. Its current implementation is best understood as a constant-density low-Mach or incompressible projection solver with optional passive species transport, optional passive sensible-enthalpy transport, and Cantera-backed thermodynamic and transport-property evaluation. The code provides a hydrodynamic and thermodynamic foundation for later reacting-flow and radiation coupling, but the current production path intentionally does not include variable-density low-Mach coupling, reactions, reaction heat release, species-diffusion enthalpy correction, or external radiation physics.

This report summarizes the solver architecture, data model, governing equations, numerical algorithms, thermodynamic conventions, MPI strategy, boundary-condition handling, profiling model, setup procedure, and known limitations.

Contents

1	Scope and Current Solver Interpretation	2
2	Enabled and Missing Physics	3
2.1	Enabled Physics	3
2.2	Missing or Reserved Physics	3
3	Repository and Module Architecture	4
3.1	Source Layout	4
3.2	Mesh Pipeline	4
3.3	MPI Decomposition	4
4	Primary Fields and State Variables	5
4.1	Flow Fields	5
4.2	Species Fields	5
4.3	Energy Fields	5
5	Governing Equations	5
5.1	Incompressible Momentum and Continuity	5
5.2	Passive Species	6
5.3	Passive Sensible Enthalpy	6

6	Finite-Volume Discretization	6
7	Projection Algorithm	6
7.1	Predictor	6
7.2	Pressure Correction	7
7.3	Correction	7
8	Species Transport	7
9	Energy and Cantera Thermodynamics	7
9.1	Sensible-Enthalpy Convention	7
9.2	Temperature Recovery	7
9.3	Option A Coupling Rule	8
9.4	Thermo Sync	8
10	Runtime Ordering	8
11	Boundary Conditions	9
12	Transport Properties and Cantera Bridge	9
13	Diagnostics, Output, and Profiling	10
14	Build and Run Procedure	10
15	Validation Strategy	10
16	Known Limitations and Future Work	11
17	Performance Characterization	11
17.1	Full Nested Profiling Tree	11
17.2	Sub-Module Cost Summary	12
17.3	Deep-Dive Analysis	12
17.3.1	Thermodynamic Bottleneck (48.4% total)	12
17.3.2	PCG Communication Sensitivity (21.1% total)	13
17.3.3	Transport Exchange Disparity	13
17.4	Recommended Optimization Priority	13
18	Summary	13

1. Scope and Current Solver Interpretation

LowMachReact-Hex currently solves constant-density incompressible flow with optional passive scalar physics. The pressure projection, momentum equation, conservative face fluxes, CFL estimate, and diagnostic mass integral use the configured flow density

$$\rho_f = \rho_{\text{params}} \quad (1)$$

In the code this value is represented by `params%rho` and stored in the transport-property container as `transport%rho`.

Cantera may compute a thermodynamic density,

$$\rho_{\text{thermo}} = \rho_{\text{Cantera}}(T, Y, p_0), \quad (2)$$

but this value is diagnostic and future-use only. It is stored as `energy%rho_thermo` and is not used in the projection or momentum equations. A future variable-density low-Mach formulation would need a different continuity constraint and pressure-correction derivation before this thermodynamic density could participate in the flow solve.

2. Enabled and Missing Physics

2.1. Enabled Physics

The current staged implementation supports:

- constant flow and projection density;
- fractional-step pressure projection for incompressible flow;
- optional passive species mass-fraction transport;
- optional passive sensible-enthalpy transport;
- Fourier heat conduction driven by ∇T ;
- Cantera evaluation of $h(T, Y, p_0)$, $T(h, Y, p_0)$, c_p , λ , mixture viscosity, species diffusivity, and diagnostic ρ_{thermo} ;
- volumetric source storage through `qrad`, with $q_{\text{rad}} > 0$ adding energy to the gas;
- VTU/PVTU output and CSV diagnostics;
- MPI-aware flat and nested profiling.

2.2. Missing or Reserved Physics

The following are not implemented in the current solver path:

- variable-density low-Mach divergence constraint;
- chemical reactions;
- reaction heat release;
- species-diffusion enthalpy correction,

$$-\nabla \cdot \left(\sum_k h_k \mathbf{J}_k \right);$$

- external radiation physics;
- radiation spectral or wavenumber decomposition;
- active use of ρ_{thermo} in the projection.

3. Repository and Module Architecture

3.1. Source Layout

The main solver source lives in `src/`. The executable program is `main.f90`, which coordinates input, mesh import, MPI setup, field initialization, the timestep loop, diagnostics, output, and finalization.

Module	Responsibility
<code>mod_kinds</code>	Precision, constants, and common fatal-error handling.
<code>mod_input</code>	Namelist parsing, parameter storage, and validation.
<code>mod_mesh_types</code>	Cell, face, patch, and mesh data structures.
<code>mod_mesh_io</code>	Native mesh-file reading.
<code>mod_bc</code>	Boundary-condition parsing and face-state evaluation.
<code>mod_fields</code>	Flow field allocation: velocity, pressure, divergence, face fluxes.
<code>mod_flow_projection</code>	Momentum prediction, pressure Poisson solve, velocity/flux correction.
<code>mod_species</code>	Passive species advection, diffusion, correction, bounding, normalization.
<code>mod_energy</code>	Sensible-enthalpy transport, temperature recovery, Cantera thermo sync.
<code>mod_transport_properties</code>	Constant or Cantera transport-property updates.
<code>mod_mpi_flow</code>	Replicated-mesh MPI ownership, exchange, gather, and reduction utilities.
<code>mod_mpi_radiation</code>	Reserved communicator/task structure for future radiation work.
<code>mod_output</code>	VTU/PVTU and CSV diagnostics output.
<code>mod_profiler</code>	Inclusive flat and nested MPI-aware timing reports.

3.2. Mesh Pipeline

The mesh workflow separates geometry generation from the solver:

1. Gmsh creates hexahedral meshes with named physical boundary surfaces.
2. A conversion tool reads the Gmsh mesh through `meshio`.
3. The converter validates the current axis-aligned hexahedral assumptions.
4. Native text files are written: `points.dat`, `cells.dat`, `faces.dat`, `patches.dat`, and optionally `periodic.dat`.
5. `mod_mesh_io` imports the native files into `mesh_t`.

3.3. MPI Decomposition

The current flow solver uses a replicated mesh and owned-cell decomposition. Every MPI rank stores the full mesh, while `mod_mpi_flow` assigns each rank a contiguous owned range of cell IDs. Computation over cell-centered unknowns is performed on owned cells, then synchronized by exchange or global collective operations.

This design is intentionally conservative for the current problem sizes and keeps the future radiation path simple: a spectral or ray-based radiation decomposition can still access the full geometry on every rank.

4. Primary Fields and State Variables

4.1. Flow Fields

The flow state includes:

- cell-centered velocity $\mathbf{u}$;
- previous velocity $\mathbf{u}^n$;
- predicted velocity $\mathbf{u}^*$;
- pressure p ;
- pressure correction ϕ ;
- velocity divergence diagnostics;
- face-centered volumetric fluxes F_f ;
- previous explicit momentum RHS for Adams–Bashforth time stepping.

The stored face flux is oriented from owner cell to neighbor cell. During cell-local transport updates, it is reoriented so that $F_{f,c} > 0$ means outflow from the currently updated cell.

4.2. Species Fields

Species are stored as mass fractions Y_k with array layout (`species`, `cell`). The number of active species is determined from `case.nml` or, in reserved reaction-discovery paths, from the Cantera mechanism. The present species equation is passive and contains no chemical source term.

4.3. Energy Fields

The transported thermal state is mixture sensible enthalpy h , not temperature. Temperature is a recovered thermodynamic variable:

$$T = T(h, Y, p_0). \quad (3)$$

The energy container also stores previous h , previous T , volumetric source q_{rad} , c_p , λ , and diagnostic ρ_{thermo} .

5. Governing Equations

5.1. Incompressible Momentum and Continuity

The incompressibility constraint is

$$\nabla \cdot \mathbf{u} = 0. \quad (4)$$

The momentum equation is advanced in fractional-step form:

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u}\mathbf{u}) = -\frac{1}{\rho_f} \nabla p + \nabla \cdot (\nu \nabla \mathbf{u}) + \mathbf{f}. \quad (5)$$

Here ρ_f is the constant projection density, ν is the kinematic viscosity used by the flow solver, and $\mathbf{f}$ is the configured body acceleration.

5.2. Passive Species

For species $k = 1, \dots, N_s$, the current model advances

$$\frac{\partial Y_k}{\partial t} + \nabla \cdot (\mathbf{u}Y_k) = \nabla \cdot (D_k \nabla Y_k), \quad (6)$$

with no reaction source:

$$\dot{\omega}_k = 0. \quad (7)$$

5.3. Passive Sensible Enthalpy

The current energy equation is

$$\frac{\partial h}{\partial t} + \nabla \cdot (\mathbf{u}h) = \frac{1}{\rho_f} \nabla \cdot (\lambda \nabla T) + \frac{q_{\text{rad}}}{\rho_f}. \quad (8)$$

Conduction is driven by ∇T , not ∇h . A positive q_{rad} increases gas sensible enthalpy.

6. Finite-Volume Discretization

For a hexahedral control volume c , let V_c be the cell volume, $f \in \partial c$ a face, A_f the face area, d_f the normal distance used for gradients, and $\mathbf{n}_{f,c}$ the outward normal relative to cell c .

For a transported scalar ψ , the solver uses upwind advection:

$$\psi_f^{\text{up}} = \begin{cases} \psi_c, & F_{f,c} \geq 0, \\ \psi_{\text{other}}, & F_{f,c} < 0, \end{cases} \quad (9)$$

where ψ_{other} is either the neighboring cell value or the boundary state.

Central differences are used for diffusive face gradients in species and thermal transport. For temperature diffusion this has the form

$$\nabla T \cdot \mathbf{n}|_f \approx \frac{T_{\text{other}} - T_c}{d_f}. \quad (10)$$

7. Projection Algorithm

The projection method proceeds in three conceptual steps.

7.1. Predictor

An intermediate velocity $\mathbf{u}^*$ is predicted without the new pressure correction:

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} = \mathcal{R}^{n,n-1} - \frac{1}{\rho_f} \nabla p^n, \quad (11)$$

where $\mathcal{R}^{n,n-1}$ represents explicit advection, diffusion, and body force terms. The solver uses Adams–Bashforth 2 when previous RHS data are available and falls back to Forward Euler on the first step.

7.2. Pressure Correction

The pressure correction potential $\phi = p^{n+1} - p^n$ is obtained from the constant-density Poisson equation

$$\nabla^2 \phi = \frac{\rho_f}{\Delta t} \nabla \cdot \mathbf{u}^*. \quad (12)$$

The matrix is applied in matrix-free form with cached geometric coefficients. The linear system is solved by preconditioned conjugate gradient with a diagonal Jacobi preconditioner. For singular pure-Neumann systems, the implementation pins the null space as needed.

7.3. Correction

The corrected velocity and pressure are

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho_f} \nabla \phi, \quad (13)$$

$$p^{n+1} = p^n + \phi. \quad (14)$$

Face fluxes are corrected consistently with the pressure potential gradient so that the updated velocity field is approximately divergence free.

8. Species Transport

Species transport uses explicit Euler time integration. Each species receives an advective flux and a diffusive flux. The diffusive fluxes are corrected at each face so that their sum is zero:

$$\sum_k J_{k,f}^{\text{corr}} = 0. \quad (15)$$

After the explicit update, mass fractions are bounded to $[0, 1]$ and renormalized locally so that

$$\sum_k Y_k = 1. \quad (16)$$

This keeps the passive species vector usable for Cantera composition calls and prevents small numerical drift from accumulating in the mixture sum.

9. Energy and Cantera Thermodynamics

9.1. Sensible-Enthalpy Convention

When Cantera thermodynamics are enabled, the stored enthalpy is not Cantera’s absolute formation-including enthalpy. The solver stores sensible enthalpy relative to a reference temperature:

$$h(T, Y, p_0) = h_{\text{abs}}^{\text{Cantera}}(T, Y, p_0) - h_{\text{abs}}^{\text{Cantera}}(T_{\text{ref}}, Y, p_0). \quad (17)$$

This convention avoids artificial heat release in non-reacting passive mixing.

9.2. Temperature Recovery

To recover temperature from stored sensible enthalpy, the code reconstructs an absolute enthalpy target:

$$h_{\text{target,abs}} = h + h_{\text{abs}}^{\text{Cantera}}(T_{\text{ref}}, Y, p_0), \quad (18)$$

then uses Cantera’s HPY inversion to find T .

9.3. Option A Coupling Rule

The code follows the Option A convention:

h is the transported thermodynamic state.

When species transport changes Y , the solver preserves h and recovers the temperature consistent with the new composition:

$$T^* = T(h, Y^{n+1}, p_0). \quad (19)$$

The solver must not recompute

$$h^* = h(T^n, Y^{n+1}, p_0) \quad (20)$$

from the old temperature and new composition, because that would add or remove sensible enthalpy without a corresponding energy flux or source.

9.4. Thermo Sync

The preferred Cantera energy-path synchronization is a combined operation:

$$(T, c_p, \lambda, \rho_{\text{thermo}}) = \text{sync}(h, Y, p_0). \quad (21)$$

The sync must preserve the transported h . If a Cantera call writes a temporary enthalpy value as part of property evaluation, the Fortran path saves and restores the transported field.

The parameter `thermo_update_interval` is reserved and must remain equal to 1 in supported runs. Internal caching can reduce redundant Cantera work, but the logical thermodynamic state must remain synchronized every energy step.

10. Runtime Ordering

The main timestep ordering is:

1. Update CFL diagnostics or dynamic timestep state when needed.
2. Refresh transport properties on `transport_update_interval`. Cantera may update μ and D_k ; flow density remains $\rho_f = \text{params}\%rho$.
3. Advance momentum and pressure projection.
4. Advance passive species, if enabled.
5. Advance passive sensible enthalpy, if enabled.
6. Recover or synchronize temperature and thermodynamic properties.
7. Write diagnostics and VTU/PVTU output on output steps.

For species-enabled Cantera thermo runs, a pre-flux energy sync is needed after species transport because Y may have changed. For species-disabled Cantera thermo runs, that pre-flux sync may be skipped because composition is unchanged and the previous post-flux sync remains valid. A post-flux sync is performed after h is updated.

11. Boundary Conditions

Boundary patches are named in the mesh and configured through `case.nml`. The code supports a legacy patch type plus field-specific overrides:

- velocity boundary type;
- pressure boundary type;
- species boundary type;
- temperature/enthalpy boundary type.

For fixed-temperature boundaries in Cantera thermo mode, boundary enthalpy is computed from the boundary temperature and boundary composition:

$$h_b = h(T_b, Y_b, p_0). \quad (22)$$

When species are enabled, Y_b comes from the species boundary condition. This is important for hot or cold fuel and oxidizer inlets, where defaulting to the interior or bath-gas composition would make boundary enthalpy inconsistent.

12. Transport Properties and Cantera Bridge

`mod.transport_properties` supports constant fallback properties and Cantera-backed transport properties. Cantera may compute:

- dynamic viscosity μ ;
- species diffusivities D_k ;
- thermal conductivity λ in the energy thermo path;
- c_p in the energy thermo path;
- diagnostic thermodynamic density.

When variable viscosity is disabled, the flow solver keeps the configured constant viscosity for validation even if Cantera species diffusivities are enabled. When variable viscosity is enabled, the kinematic viscosity is computed from Cantera μ and the constant flow density:

$$\nu = \frac{\mu}{\rho_f}. \quad (23)$$

The C++ bridge maps solver species names to Cantera mechanism species names, constructs full Cantera mass-fraction vectors, fills missing bath gas when needed, and caches repeated thermo or transport evaluations conservatively.

13. Diagnostics, Output, and Profiling

The solver writes global diagnostics and ParaView-compatible visualization files. Output may include velocity, pressure, divergence, temperature, enthalpy, q_{rad} , c_p , λ , thermal diffusivity, ρ_{thermo} , species mass fractions, species diffusivities, and cell IDs.

The profiler reports inclusive wall-clock timing. Flat profiler rows are not additive when nested timers are enabled. Common top-level timers include:

- `Transport_Update`;
- `Projection_Step`;
- `Species_Transport`;
- `Energy_Transport`;
- `Diagnostics_Write_Flow`;
- `Diagnostics_Write_Energy`;
- `Output_Write_VTU`.

Cantera energy sync timers include `Energy_Cantera_PreSync` and `Energy_Cantera_PostSync`.

14. Build and Run Procedure

The project expects a Conda environment created from `environment.yml` and an available OpenMPI compiler/runtime.

```
conda env create -f environment.yml
conda activate react_env

# Load OpenMPI if your system uses environment modules.
module load openmpi

# Build.
make BUILD=release

# Run.
mpirun -np 4 ./lowmach_react_hex cases/channel_flow/case.nml
```

The Makefile creates a compiled binary under `build/` and a top-level `lowmach_react_hex` launcher. Users should run the launcher. It prepends the active Conda library directory so Cantera and the C++ runtime are resolved from the environment.

15. Validation Strategy

Recommended validation proceeds from low-risk to high-risk configurations:

1. Build in debug mode and run a zero- or few-step startup smoke test.
2. Validate single-rank incompressible flow on a simple case.
3. Repeat the same case with multiple MPI ranks and compare diagnostics.

4. Enable passive species transport with constant diffusivity.
5. Enable constant-property energy transport.
6. Enable Cantera thermo without transported species.
7. Enable species-coupled Cantera thermo and verify the Option A behavior.
8. Compare profiler output before and after performance changes.

For each validation run, keep the case file, rank count, build mode, terminal output, diagnostics CSV files, and representative visualization output.

16. Known Limitations and Future Work

The most important current limitation is the deliberate constant-density projection formulation. Future variable-density low-Mach support should not be added by simply replacing ρ_f with ρ_{thermo} ; it requires a redesigned divergence constraint and pressure equation.

Additional future work includes:

- reaction source terms and reaction heat release;
- species-diffusion enthalpy correction;
- radiation physics coupled through q_{rad} ;
- improved mesh partitioning beyond replicated full-mesh storage;
- higher-order bounded advection for scalar transport;
- broader regression tests across energy/species/Cantera modes.

17. Performance Characterization

The solver’s performance is dominated by the coupling between thermodynamic property synchronization and the iterative pressure correction solver. Based on a 16,000-step benchmark on a multi-rank hexahedral mesh (total wall time 83.6 s avg across ranks), the following granular profiling tree was collected.

17.1. Full Nested Profiling Tree

Region	Tree	Calls	Avg(s)	Avg%

Total_Simulation	1	83.564	100.00	
MPI_Communication	1	0.005	0.01	
Transport_Update	16	0.186	0.22	
Transport_Setup	16	0.000	0.00	
Transport_Cantera_Call	16	0.074	0.09	
Transport_Exchange_Diff	16	0.112	0.13	
MPI_Communication	16	0.112	0.13	
Projection_Step	16000	33.994	40.68	
Projection_Momentum_RHS	16000	1.732	2.07	
Projection_AB2	16000	0.169	0.20	
Projection_Predict_Flux	16000	2.105	2.52	

```

PredictFlux_Compute 16000 0.408 0.49
PredictFlux_Balance 16000 1.194 1.43
MPI_Communication 16000 0.771 0.92
Projection_Poisson_RHS 16000 0.133 0.16
Projection_PCG 16000 28.053 33.57
MPI_Communication 6629753 17.609 21.07
Pressure_Matvec 2199251 5.786 6.92
Projection_Pressure_Update 16000 0.074 0.09
Projection_Correction 16000 1.370 1.64
Species_Transport 16000 1.531 1.83
MPI_Communication 16000 0.209 0.25
Energy_Transport 16000 47.180 56.46
Energy_Cantera_PreSync 16000 21.847 26.14
Energy_PreFlux_Exchange 16000 3.794 4.54
MPI_Communication 32000 3.773 4.52
Energy_Flux_Update 16000 0.653 0.78
Energy_Cantera_PostSync 16000 18.591 22.25
Energy_Final_Exchange 16000 2.029 2.43
MPI_Communication 48000 2.002 2.40

```

17.2. Sub-Module Cost Summary

Sub-Region	Avg s	%	Technical Observation
Energy: Cantera PreSync	21.85	26.1%	HPY inversion per cell, pre-flux.
Energy: Cantera PostSync	18.59	22.3%	Property refresh after h update.
Projection: PCG MPI	17.61	21.1%	Global dot products and halo exchanges.
Projection: Matvec	5.79	6.9%	Matrix-free Laplacian (efficient).
Energy: PreFlux Comm	3.77	4.5%	MPI exchange of T , h , c_p , λ .
Energy: Final Comm	2.00	2.4%	Post-update thermodynamic field sync.
Projection: Predictor	2.11	2.5%	Momentum RHS and flux computation.
Species: Transport+Comm	1.53	1.8%	Explicit advection-diffusion (low cost).

17.3. Deep-Dive Analysis

17.3.1 Thermodynamic Bottleneck (48.4% total)

The `Energy_Cantera_Sync` cycle is the single dominant cost. Pre-flux state recovery (HPY inversion) consumes **26.1%** and post-flux property refreshment consumes **22.3%** of total wall time. The cost is nearly symmetric because both operations call the same per-cell Cantera HPY solve. This thermodynamic compute cost is approximately **20×** the cost of the entire hydrodynamic predictor step.

Optimization Opportunity: Cantera

Batch cell-state HPY inversions across the mesh rather than calling Cantera cell-by-cell. Alternatively, amortize the cost by caching thermo properties when T and Y changes are below a tolerance threshold.

17.3.2 PCG Communication Sensitivity (21.1% total)

The Pressure Poisson solver (PCG) is executed 16,000 times with a total of 6,629,753 `MPI_Communication` calls. Of the **33.6%** total time in `Projection.PCG`, over **63%** (21.1% of total) is pure MPI overhead. This identifies the solver as *interconnect-latency bound*: the bottleneck is the global dot products required by the CG algorithm, not the arithmetic.

Optimization Opportunity: PCG

Replace the standard PCG with a pipelined or CA-CG variant to overlap communication with computation. Alternatively, coarsen the problem with AMG to reduce iteration count and thus global reduction frequency.

17.3.3 Transport Exchange Disparity

Species transport incurs only **0.25%** MPI overhead, whereas energy transport incurs **6.9%** (PreFlux + Final exchanges combined). The cause is the number of fields that must be synchronized: species exchanges only Y_k , while energy exchanges require T , h , c_p , and λ for boundary-cell consistency with Cantera.

17.4. Recommended Optimization Priority

1. **Batch Cantera HPY Inversions:** Profile-directed batching of per-cell thermo calls is expected to yield the largest wall-time reduction (target: reduce 48.4% cost by $\sim 50\%$).
2. **Pipelined or Communication-Avoiding PCG:** Reduces the frequency of global reductions, which currently account for 63% of the Poisson solve time.
3. **Fused Energy Exchange:** Combine the PreFlux and Final MPI exchanges into a single non-blocking operation where possible.

18. Summary

LowMachReact-Hex is currently a constant-density projection solver with well-defined passive species and sensible-enthalpy transport extensions. The critical state convention is:

- h (sensible enthalpy relative to T_{ref}) is the transported thermodynamic scalar.
- T is a *recovered* variable, obtained via Cantera HPY inversion from (h, Y, p_0) .
- ρ_{thermo} (Cantera thermodynamic density) is *diagnostic only* and is never used in the projection.

These conventions make the solver internally consistent while preserving a clear path toward future extensions. The performance analysis identifies Cantera thermodynamic synchronization (48.4%) and PCG MPI communication (21.1%) as the two primary targets for optimization before scaling to larger meshes or longer runs.

Future Architecture Warning

Do not activate variable-density low-Mach coupling by simply substituting ρ_{thermo} for ρ_f in the projection. This requires a full redesign of the divergence constraint and pressure-correction equation.